

Laboratorio 5 - Esercizi sulla concorrenza

L'obiettivo di questo laboratorio è di prendere dimestichezza con alcuni dei costrutti che Rust mette a disposizione per la programmazione concorrente. Per fare questo è strutturato in 6 esercizi, di complessità crescente, ciascuno isolato l'uno dall'altro. Nella cartella `src/` troverete 6 file sorgente, uno per esercizio, con al loro interno il codice di partenza (boilerplate) di ciascun esercizio. Per verificare che la vostra implementazione sia corretta potete utilizzare i testi forniti con ciascun esercizio, eseguendo su terminale il comando `cargo test <nome esercizio>`, ad esempio `cargo test es1_parallel_sum`. Invece, per eseguire i test di tutti gli esercizi, potete semplicemente eseguire `cargo test` senza alcun argomento.

Esercizio 1 - Somma parallela di un vettore

Si vuole calcolare la somma di tutti gli elementi di un vettore di `i64` sfruttando più thread. Scrivere una funzione

Rust

```
fn parallel_sum(data: Vec<i64>, n_threads: usize) -> i64
```

che suddivida il vettore in `n_threads` porzioni di dimensione uguale, crei un thread per ciascuna porzione, calcoli la somma parziale di quella porzione all'interno del thread corrispondente e infine restituisca la somma totale ottenuta combinando i risultati parziali.

Requisiti:

- Ciascun thread deve restituire la propria somma parziale tramite il valore di ritorno della chiusura passata a `thread::spawn`, che verrà recuperato attraverso `join()`.
- Si presti attenzione al caso in cui la lunghezza del vettore non sia un multiplo di `n_threads`: tutti gli elementi devono comunque essere considerati una sola volta.
- Si gestisca correttamente il caso `n_threads == 0` (errore) e il caso in cui il vettore sia più corto di `n_threads`.

Domande:

- Quanto costa (in termini di tempo/cicli macchina) creare N thread?
- Quanto costa sommare sequenzialmente i valori?
- Quanto deve essere grande il vettore per avere un vantaggio nell'esecuzione parallela?
- Cosa cambierebbe se invece di calcolare la somma dei valori si effettuasse un'operazione più complessa?

Esercizio 2 - Conteggio parallelo con stato condiviso

Dato un grande vettore di `u32`, si vuole determinare quanti elementi siano divisibili per un valore `k` fornito dall'utente. Il calcolo deve essere svolto da `n_threads` thread che cooperano aggiornando un **contatore condiviso**.

Scrivere una funzione

Rust

```
fn count_divisible(  
    data: Vec<u32>,  
    k: u32,  
    n_threads: usize  
) -> usize
```

che soddisfi i seguenti requisiti:

Requisiti:

- Il contatore condiviso deve essere implementato come `Arc<Mutex<usize>>`.
- Ogni thread elabora la propria porzione di vettore e, **al termine**, aggiorna una sola volta il contatore globale aggiungendo il proprio conteggio parziale.

Si implementi la funzione

Rust

```
fn count_divisible_atomic(  
    data: Vec<u32>,  
    k: u32,  
    n_threads: usize  
) -> usize
```

che gestisca il contatore condiviso con una variabile di tipo `AtomicUsize` e si verifichi che restituisce gli stessi risultati della precedente.

Domande:

- Cosa cambia, in termini di tempo di esecuzione, se il contatore condiviso viene aggiornato ad ogni rilevamento?
- C'è differenza di prestazioni c'è tra le due implementazioni in questo caso?

Esercizio 3 - Statistiche concorrenti con `RwLock`

Si vuole realizzare una piccola struttura dati che mantenga statistiche in tempo reale su una sequenza di campioni numerici prodotti da più thread "produttori", e che possa essere interrogata in modo efficiente da molti thread "lettori".

Definire una struct `SharedStats` che incapsuli uno stato condiviso con le statistiche e che esponga i seguenti metodi pubblici:

- `new() -> Self`
- `add_sample(&self, value: f64)` - aggiorna le statistiche con un nuovo campione.
- `count(&self) -> usize` - restituisce il numero di campioni ricevuto
- `mean(&self) -> Option<f64>` - restituisce `None` se non ci sono campioni.
- `min(&self) -> Option<f64>`
- `max(&self) -> Option<f64>`
- `snapshot(&self) -> Option<(usize, f64, f64, f64)>` - restituisce in un'unica operazione conteggio, media, minimo e massimo, garantendo che i quattro valori siano coerenti tra loro.

Domande:

- Perché è necessario introdurre il metodo `snapshot()` e non basta chiamare in sequenza i metodi relativi a media/massimo/minimo?
- Si possono, in questo esercizio, utilizzare dei campi atomici oppure occorre una sincronizzazione complessiva della struttura dati?
- Che differenza di prestazioni ci sarebbe se invece di un `RwLock` si usasse un `Mutex`?

Esercizio 4 - Word frequency multi-file (mini map-reduce)

Si vuole realizzare un programma che, dato un elenco di percorsi di file di testo, calcoli la **frequenza globale** di ciascuna parola (case-insensitive) presente al loro interno, considerando tutti i file insieme. Il programma deve sfruttare il parallelismo tra file: ciascun file deve essere elaborato da un thread distinto.

Realizzare le seguenti funzioni:

```
Rust
fn word_frequencies(paths: Vec<String>) -> HashMap<String, usize>
fn top_k(
    freqs: &HashMap<String, usize>,
    k: usize
) -> Vec<(String, usize)>
```

Requisiti:

- Ogni thread legge e processa un singolo file, costruendo una `HashMap<String, usize>` locale con la frequenza delle parole presenti **solo in quel file**.
- Una volta terminata la propria computazione, ciascun thread deve fondere la propria mappa locale in una mappa globale condivisa, protetta da un `Mutex`. La fusione deve avvenire **una sola volta per thread**, non parola per parola.
- Se l'apertura o la lettura di un file fallisce, il thread corrispondente non deve far terminare il programma: deve invece restituire un risultato vuoto (o un errore gestito) senza interrompere gli altri thread.
- La funzione `top_k` deve restituire le `k` parole più frequenti, ordinate per frequenza decrescente.

Domande:

- Cosa cambierebbe se invece di aggiornare le statistiche condivise al termine dell'analisi di ciascun file, queste venissero aggiornate puntualmente?
- Ci sarebbe un beneficio ad usare forme di sincronizzazione differenti?

Esercizio 5 - Mini grep multi-file multi-thread

Si vuole realizzare una versione semplificata del comando Unix `grep`: dato un pattern (stringa letterale, **non** regex) e un elenco di file, il programma deve cercare in parallelo tutte le righe che contengono il pattern e produrre un risultato consolidato.

Definire la struct e la funzione:

```
Rust
struct Match { /* campi */ }

fn parallel_grep(
    pattern: String,
    paths: Vec<String>,
    n_workers: usize
) -> Vec<Match>
```

che soddisfi i seguenti requisiti.

Requisiti:

- È disponibile una **coda condivisa di file da elaborare**, realizzata come `Arc<Mutex<Vec<String>>>`. Inizialmente la coda contiene tutti i percorsi passati come argomento.
- Vengono creati `n_workers` thread. Ciascun thread ripete il seguente ciclo finché ci sono file da elaborare:
 1. acquisisce il lock sulla coda,
 2. estrae (con `pop`) un file dalla coda - se la coda è vuota, il thread termina,
 3. **rilascia il lock** prima di iniziare a elaborare il file
 4. apre il file, lo legge riga per riga, e per ogni riga che contiene il pattern aggiunge un `Match` a un vettore locale,
 5. al termine del file, accoda il proprio vettore locale di match a un **vettore globale di risultati**, anch'esso protetto da `Mutex`.
- L'errore di apertura di un file non deve interrompere il programma: il thread che lo riscontra deve passare al file successivo.

Domande:

- Perché è importante rilasciare il lock sulla coda di file *prima* di iniziare a leggere il file estratto? Cosa succederebbe se il lock fosse tenuto per tutta la durata della lettura?
- Sarebbe stato corretto usare `Arc<RwLock<Vec<String>>>` al posto di `Arc<Mutex<Vec<String>>>` per la coda dei file? Perché?

Esercizio 6 - Cache condivisa con politica di scadenza

Si vuole realizzare una cache thread-safe utilizzabile da molti thread in lettura (frequente) e da pochi thread in scrittura. La cache associa chiavi di tipo `String` a valori di tipo generico `V`. Ogni voce della cache è associata a un **istante di inserimento**; le voci più vecchie di una soglia configurabile (TTL - time-to-live) sono considerate scadute e devono essere rimosse.

Definire la struct generica

```
Rust
pub struct Cache<V> { /* campi */ }
```

con i seguenti metodi pubblici:

- `pub fn new(ttl: Duration) -> Self`
- `pub fn insert(&self, key: String, value: V)` - inserisce o sovrascrive la voce associata a `key`, aggiornando l'istante di inserimento.

- `pub fn get(&self, key: &str) -> Option<V>` - restituisce il valore associato a `key` se è presente e *non è scaduto*; restituisce `None` altrimenti. Una voce scaduta letta tramite `get` deve essere rimossa dalla cache (lazy eviction).
- `pub fn len(&self) -> usize` - numero di voci attualmente presenti (incluse le scadute non ancora rimosse).
- `pub fn purge_expired(&self) -> usize` - rimuove tutte le voci scadute e restituisce il numero di voci eliminate.

Requisiti:

- La struttura interna principale (la mappa chiave -> (valore, istante)) deve essere protetta da un `RwLock`, sfruttando il fatto che le letture sono molto più frequenti delle scritture.
- La struct `Cache<V>` deve essere utilizzabile da più thread tramite `Arc`. Ragionare su quali tratti deve soddisfare `V` perché ciò sia possibile, e dichiararli esplicitamente nei vincoli di tipo.

Domande:

- Se volessimo che le chiavi scadute fossero rimosse in modo attivo invece che passivo, ovvero che venissero cancellate il prima possibile dopo la loro scadenza invece che quando vi è un tentativo di estrazione con `get`, come affrontereste il problema?
- Come gestireste invece una cache di tipo `LRU` ? Quali modifiche dovreste applicare al vostro programma per cambiare il comportamento della cache attuale?